MAR BASELIOS

COLLEGE OF ENGINEERING AND TECHNOLOGY

(Autonomous College affiliated to APJ Abdul Kalam Technological University)

MAR IVANIOS VIDYANAGAR, THIRUVANANTHAPURAM – 15

Department of Electrical & Electronics Engineering

23EEP20B - SIMULATION LAB MANUAL

Name:

Register number:

Department:

Prepared by :

Dr.Dishore SV,Assistant Professor,Electrical and Electronics Engineering,Mar Baselios College of Engineering and Technology

LIST OF EXPERIMENTS			
EX.NO		**DATE**	**Signature**
1.	Computation of DC Biases in a Resistor Bridge Using PYSPICE		
2.	Simulation of A Current-Controlled Current Source (CCCS) CIRCUIT Using PYSPICE		
3.	Simulation of A Voltage-Controlled Voltage Source (VCVS)CIRCUIT Using PYSPICE		
4.	Simulating Natural Response of RC Circuits Using PYSPICE		
5.	Simulating Step Response of RC Circuits Using PYSPICE		
6.	Simulation of a pacemaker circuit Using PYSPICE		
7.	Verification of Truth Tables of Logic Gates Using Multisim		
8.	Simulation of Half Adder and Full Adder Circuit in Multisim		
9.	Implementation of Boolean Function in SOP and POS Form Using Multisim		
10.	Implementation of Combinational circuit using minimum number of NAND Gates Using Multisim		
11.	Simulate the V-I characteristics of Diode Using MATLAB Simulink		
12.	Simulate a voltage regulator using Zener diode in MATLAB Simulink		
13.	Simulate a Half wave rectifier circuit in MATLAB Simulink		
14.	Simulate a full wave rectifier circuit in MATLAB Simulink		

EXPERIMENT-1 **Date:**

COMPUTATION OF DC BIASES IN A RESISTOR BRIDGE USING PYSPICE

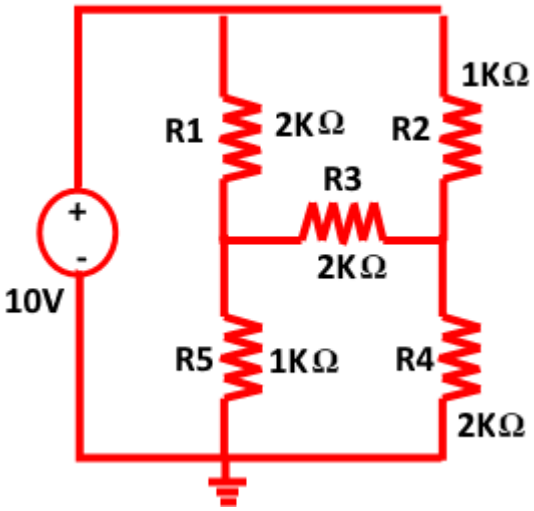
Aim:

To compute the DC biases at various nodes in a resistor bridge circuit using a Python-based circuit simulation.

Apparatus Required:

1. Python programming environment with PySpice library installed
2. Computer with an internet connection to set up the Python environment

Circuit diagram:



Procedure:

1. **Setup the Python Environment:**
   - Ensure that Python and PySpice library are installed on your computer. If not, install PySpice using the command:

     pip install PySpice

2. **Write the Circuit Code:**
   - Open a new text document and write the following code to define the resistor bridge circuit and compute the DC biases:

     ```
     import PySpice.Logging.Logging as Logging
     logger = Logging.setup_logging()
     ```

```
from PySpice.Spice.Netlist import Circuit
from PySpice.Unit import *

# Define the circuit
circuit = Circuit('Resistor Bridge')

# Add components
circuit.V('input', 1, circuit.gnd, 10@u_V)
circuit.R(1, 1, 2, 2@u_kΩ)
circuit.R(2, 1, 3, 1@u_kΩ)
circuit.R(3, 2, circuit.gnd, 1@u_kΩ)
circuit.R(4, 3, circuit.gnd, 2@u_kΩ)
circuit.R(5, 3, 2, 2@u_kΩ)

# Perform the simulation
simulator = circuit.simulator(temperature=25, nominal_temperature=25)
analysis = simulator.operating_point()

# Print the results
for node in analysis.nodes.values():
    print('Node {}: {:4.1f} V'.format(str(node), float(node))) # Format value +
unit
```

Save the text document in .py format.

3. **Run the Simulation:**
   - Execute the Python script to run the simulation. This will compute the DC biases at each node in the resistor bridge.
4. **Analyze the Results:**
   - Observe the output of the script, which will display the voltage at each node in the resistor bridge.

Result:

EXPERIMENT-2 **Date:**

SIMULATION OF A CURRENT-CONTROLLED CURRENT SOURCE (CCCS) CIRCUIT USING PYSPICE

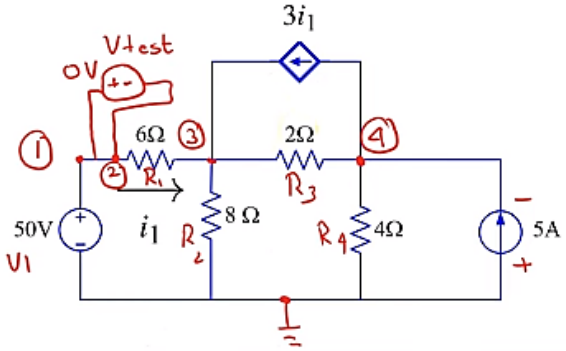
Aim:

To analyze the behavior of a circuit containing resistors, voltage sources, and a current-controlled current source (CCCS) by computing the DC biases and currents through various elements.

Apparatus Required:

1. Python programming environment with PySpice library installed
2. Computer with an internet connection to set up the Python environment

Circuit diagram:



Procedure:

1. **Setup the Python Environment:**
   - Ensure that Python and the PySpice library are installed on your computer. If not, install PySpice using the command:

     pip install PySpice

2. **Write the Circuit Code:**
   - Open a new text document and write the following code to define the circuit with resistors, voltage sources, and a CCCS:

```
import PySpice.Logging.Logging as Logging
logger = Logging.setup_logging()

from PySpice.Spice.Netlist import Circuit
from PySpice.Unit import *

# Define the circuit
circuit = Circuit('Circuit with CCCS')

# Add resistors
circuit.R(1, 2, 3, 6@u_Ω)
circuit.R(2, 3, circuit.gnd, 8@u_Ω)
```

```
circuit.R(3, 3, 4, 2@u_Ω)
circuit.R(4, 4, circuit.gnd, 4@u_Ω)

# Add voltage sources
circuit.V(1, 1, circuit.gnd, 50@u_V)
circuit.V('test', 1, 2, 0@u_V)

# Add current source
circuit.I(1, circuit.gnd, 4, 5@u_A)

# Add CCCS (F element)
circuit.F(1, 4, 3, 'Vtest', 3)

# Perform the simulation
simulator = circuit.simulator(temperature=25, nominal_temperature=25)
analysis = simulator.operating_point()

# Print the results
for node in analysis.nodes.values():
    print('Node {}: {:4.1f} V'.format(str(node), float(node)))

for branch in analysis.branches.values():
    print('Branch {}: {:5.2f} A'.format(str(branch), float(branch)))
```

Save the text document in .py format.

3. **Run the Simulation:**
   - Execute the Python script to run the simulation. This will compute the DC biases and currents through various components in the circuit.
4. **Analyze the Results:**
   - Observe the output of the script, which will display the voltage at each node and the current through each branch in the circuit.

Result:

EXPERIMENT-3 **Date:**

SIMULATION OF A VOLTAGE-CONTROLLED VOLTAGE SOURCE (VCVS)CIRCUIT USING PYSPICE

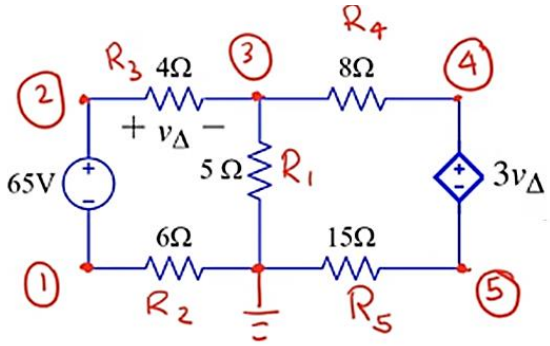
Aim:

To analyze the behavior of a circuit containing resistors, voltage sources, and a voltage-controlled voltage source (VCVS) by computing the DC biases and currents through various elements.

Apparatus Required:

1. Python programming environment with PySpice library installed
2. Computer with an internet connection to set up the Python environment

Circuit diagram:



Procedure:

1. **Setup the Python Environment:**
   - Ensure that Python and the PySpice library are installed on your computer. If not, install PySpice using the command:

     pip install PySpice

2. **Write the Circuit Code:**
   - Open a new text document and write the following code to define the circuit with resistors, voltage sources, and a VCVS:

     ```
     import PySpice.Logging.Logging as Logging
     logger = Logging.setup_logging()

     from PySpice.Spice.Netlist import Circuit
     from PySpice.Unit import *

     # Define the circuit
     circuit = Circuit('Circuit with VCVS')


     # Add resistors
     ```

```
circuit.R(1, 3, circuit.gnd, 5@u_Ω)
circuit.R(2, 1, circuit.gnd, 6@u_Ω)
circuit.R(3, 2, 3, 4@u_Ω)
circuit.R(4, 3, 4, 8@u_Ω)
circuit.R(5, 5, circuit.gnd, 15@u_Ω)
# Add voltage source
circuit.V(1, 2, 1, 65@u_V)

# Add VCVS (E element)
circuit.VCVS(1, 4, 5, 2, 3, 3)

# Perform the simulation
simulator = circuit.simulator(temperature=25, nominal_temperature=25)
analysis = simulator.operating_point()

# Print the results
for node in analysis.nodes.values():
    print('Node {}: {:4.1f} V'.format(str(node), float(node)))

for branch in analysis.branches.values():
    print('Branch {}: {:5.2f} A'.format(str(branch), float(branch)))
```

Save the text document in .py format.

3. **Run the Simulation:**
    - Execute the Python script to run the simulation. This will compute the DC biases and currents through various components in the circuit.
4. **Analyze the Results:**
    - Observe the output of the script, which will display the voltage at each node and the current through each branch in the circuit.

Result:

EXPERIMENT-4 **Date:**

SIMULATION OF NATURAL RESPONSE ANALYSIS OF AN RC CIRCUIT USING PYSPICE

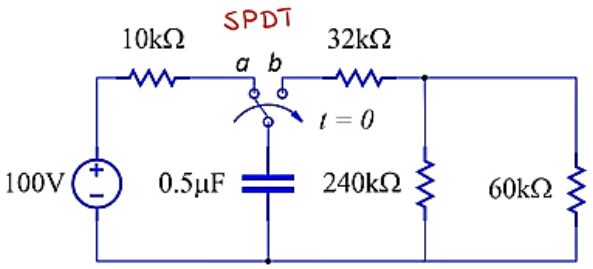
Aim:

To analyze the natural response of an RC circuit with voltage-controlled switches by observing the voltage across a capacitor and the current through the circuit after switching.

Apparatus Required:

1. Python programming environment with PySpice and Matplotlib libraries installed
2. Computer with an internet connection to set up the Python environment

Circuit diagram:



Procedure:

1. **Setup the Python Environment:**
   - Ensure that Python, PySpice, and Matplotlib libraries are installed on your computer. If not, install them using the following commands:

     pip install PySpice matplotlib

2. **Write the Circuit Code:**
   - Open a text document and write the following code to define the RC circuit with voltage-controlled switches and simulate its natural response:

```
import matplotlib.pyplot as plt
from matplotlib.widgets import Cursor
import numpy as np

import PySpice.Logging.Logging as Logging
logger = Logging.setup_logging()

from PySpice.Doc.ExampleTools import find_libraries
from PySpice.Probe.Plot import plot
from PySpice.Spice.Library import SpiceLibrary
from PySpice.Spice.Netlist import Circuit
from PySpice.Unit import *

import math
```

```
from engineering_notation import EngNumber

#################################################################
######
# CIRCUIT NETLIST
circuit = Circuit('Natural Response of RC Circuit')

steptime = 1@u_us
switchingtime = 50@u_ms
finaltime = 250@u_ms

circuit.model('switch', 'SW', Ron=1@u_mΩ, Roff=1@u_GΩ)

circuit.PulseVoltageSource(2, 'posa', circuit.gnd, initial_value=1,
pulsed_value=-1,
                    pulse_width=finaltime, period=finaltime,
delay_time=switchingtime)
circuit.R('testa', 'posa', circuit.gnd, 1@u_kΩ)

circuit.PulseVoltageSource(3, 'posb', circuit.gnd, initial_value=-1,
pulsed_value=1,
                    pulse_width=finaltime, period=finaltime,
delay_time=switchingtime)
circuit.R('testb', 'posb', circuit.gnd, 1@u_kΩ)

circuit.V(1, 1, circuit.gnd, 100@u_V)
circuit.R(1, 1, 'a', 10@u_kΩ)
circuit.VoltageControlledSwitch(1, 'a', 'output', 'posa', circuit.gnd,
model='switch')
circuit.C(1, 'output', circuit.gnd, 0.5@u_uF)
circuit.VoltageControlledSwitch(2, 'output', 'b', 'posb', circuit.gnd,
model='switch')
circuit.R(2, 'b', 2, 32@u_kΩ)
circuit.R(3, 2, circuit.gnd, 240@u_kΩ)
circuit.R(4, 2, circuit.gnd, 60@u_kΩ)

simulator = circuit.simulator(temperature=25, nominal_temperature=25)
ic = 0@u_V
simulator.initial_condition(output=ic)
analysis = simulator.transient(step_time=steptime, end_time=finaltime)

#################################################################
########
# THEORETICAL CALCULATIONS

# Time constant after switching
tau = (circuit.C1.capacitance) * (circuit.R2.resistance +
                        (circuit.R3.resistance) * (circuit.R4.resistance)
                        / (circuit.R3.resistance + circuit.R4.resistance))
print('tau={0}'.format(EngNumber(tau.value)))
```

```
#####################################################################
########
# PLOTTING COMMANDS
figure = plt.subplots(figsize=(11, 6))
axe = plt.subplot(211)
plt.title('Switch activations')
plt.ylabel('Voltage [V]')
plt.grid()
plot(analysis['posa'], axis=axe)
plot(analysis['posb'], axis=axe)
plt.legend(('position a', 'position b'), loc=(.05, .1))
axe.set_yticks([-1, 0, 1])
axe = plt.subplot(212)
plt.title('Voltage across Capacitor')
plt.xlabel('Time [s]')
plt.ylabel('Voltage [V]')
plt.grid()
plot(analysis['output'], axis=axe, color='black')
plot(analysis['2'], axis=axe, color='magenta')
plt.legend(('Output', 'v_R4'), loc=(.05, .1))
axe.set_yticks([0, 100 - (100 / math.exp(1)), 100 / math.exp(1), 100])
cursor = Cursor(axe, useblit=True, color='red', linewidth=1)

figure, axe = plt.subplots(figsize=(11, 6))
plt.title('Currents')
plt.xlabel('Time [s]')
plt.ylabel('Current [A]')
plt.grid()
plot((analysis['1'] - analysis['a']) / circuit.R1.resistance, axis=axe,
color='black')
plot(analysis['2'] / circuit.R4.resistance, axis=axe, color='magenta')
plot((analysis['b'] - analysis['2']) / circuit.R2.resistance, axis=axe, color='blue')
plt.legend(('i_R1', 'i_R4', 'i_R2'), loc=(.05, .1))

plt.show()
```

Save the text document in .py format.

3. **Run the Simulation:**
   - Execute the Python script to run the simulation. This will compute the transient response of the RC circuit and generate plots showing the voltage across the capacitor and the current through the circuit after switching.
4. **Analyze the Results:**
   - Observe the output of the script, which will display the following plots:
     - Switch activations over time
     - Voltage across the capacitor over time
     - Currents through various resistors over time

Result:

EXPERIMENT-5 **Date:**

SIMULATION OF STEP RESPONSE ANALYSIS OF AN RC CIRCUIT USING PYSPICE

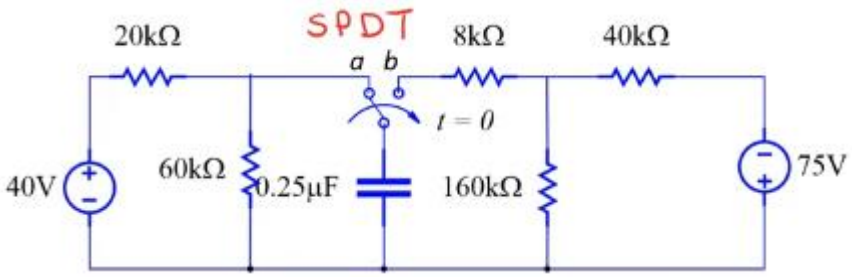
Aim:

To analyze the step response of an RC circuit with voltage-controlled switches by observing the voltage across a capacitor and the current through the circuit after switching.

Apparatus Required:

1. Python programming environment with PySpice and Matplotlib libraries installed
2. Computer with an internet connection to set up the Python environment

Circuit diagram:



Procedure:

1. **Setup the Python Environment:**
   - Ensure that Python, PySpice, and Matplotlib libraries are installed on your computer. If not, install them using the following commands:

     pip install PySpice matplotlib

2. **Write the Circuit Code:**
   - Open a text document and write the following code to define the RC circuit with voltage-controlled switches and simulate its step response:

```
import matplotlib.pyplot as plt
from matplotlib.widgets import Cursor
import numpy as np

import PySpice.Logging.Logging as Logging
logger = Logging.setup_logging()

from PySpice.Doc.ExampleTools import find_libraries
from PySpice.Probe.Plot import plot
from PySpice.Spice.Library import SpiceLibrary
from PySpice.Spice.Netlist import Circuit
from PySpice.Unit import *
```

```
import math
from engineering_notation import EngNumber

##################################################################
######
# CIRCUIT NETLIST
circuit = Circuit('Step Response of RC Circuit')

steptime = 1@u_us
switchingtime = 20@u_ms
finaltime = 80@u_ms
circuit.model('switch', 'SW', Ron=1@u_mΩ, Roff=1@u_GΩ)
circuit.PulseVoltageSource(3, 'posa', circuit.gnd, initial_value=1,
pulsed_value=-1,pulse_width=finaltime, period=finaltime,
delay_time=switchingtime)
circuit.R('testa', 'posa', circuit.gnd, 1@u_kΩ)
circuit.PulseVoltageSource(4, 'posb', circuit.gnd, initial_value=-1,
pulsed_value=1, pulse_width=finaltime, period=finaltime,
delay_time=switchingtime)
circuit.R('testb', 'posb', circuit.gnd, 1@u_kΩ)
circuit.V(1, 1, circuit.gnd, 40@u_V)
circuit.R(1, 1, 'a', 20@u_kΩ)
circuit.R(2, 'a', circuit.gnd, 60@u_kΩ)
circuit.VoltageControlledSwitch(1, 'a', 'output', 'posa', circuit.gnd,
model='switch')
circuit.C(1, 'output', circuit.gnd, 0.25@u_uF)
circuit.VoltageControlledSwitch(2, 'output', 'b', 'posb', circuit.gnd,
model='switch')
circuit.R(3, 'b', 2, 8@u_kΩ)
circuit.R(4, 2, circuit.gnd, 160@u_kΩ)
circuit.R(5, 2, 3, 40@u_kΩ)
circuit.V(2, circuit.gnd, 3, 75@u_V)
simulator = circuit.simulator(temperature=25, nominal_temperature=25)
ic = 0@u_V
simulator.initial_condition(output=ic)
analysis = simulator.transient(step_time=steptime, end_time=finaltime)
# THEORETICAL CALCULATIONS
# Time constant after switching
tau = (circuit.C1.capacitance) * (circuit.R3.resistance +
                          (circuit.R4.resistance) * (circuit.R5.resistance)
                          / (circuit.R4.resistance + circuit.R5.resistance))
print('tau={0}'.format(EngNumber(tau.value)))
# PLOTTING COMMANDS
figure = plt.subplots(figsize=(11, 6))
axe = plt.subplot(211)
plt.title('Switch activations')
plt.ylabel('Voltage [V]')
plt.grid()
plot(analysis['posa'], axis=axe)
plot(analysis['posb'], axis=axe)
```

```
plt.legend(('position a', 'position b'), loc=(.05, .1))
axe.set_yticks([-1, 0, 1])
axe = plt.subplot(212)
plt.title('Voltage across Capacitor')
plt.xlabel('Time [s]')
plt.ylabel('Voltage [V]')
plt.grid()
plot(analysis['output'], axis=axe, color='black')
axe.set_yticks([-60, 0, 30])
cursor = Cursor(axe, useblit=True, color='red', linewidth=1)
figure, axe = plt.subplots(figsize=(11, 6))
plt.title('Capacitor current after switching')
plt.xlabel('Time [s]')
plt.ylabel('Current [A]')
plt.grid()
plot((analysis['b'] - analysis['2']) / circuit.R3.resistance, axis=axe,
color='black')
plt.show()
```

Save the text document in .py format.

3. **Run the Simulation:**
    - Execute the Python script to run the simulation. This will compute the transient response of the RC circuit and generate plots showing the voltage across the capacitor and the current through the circuit after switching.
4. **Analyze the Results:**
    - Observe the output of the script, which will display the following plots:
        - Switch activations over time
        - Voltage across the capacitor over time
        - Capacitor current after switching

Result:

EXPERIMENT-6 **Date:**

SIMULATION OF A PACEMAKER CIRCUIT IN PYSPICE

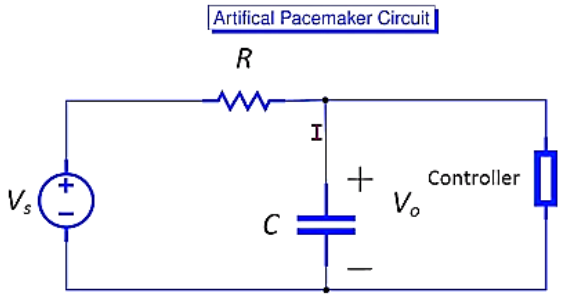
Aim:

To simulate the operation of a pacemaker circuit and determine the required resistor value to achieve a desired heart rate.

Apparatus Required:

1. Python programming environment with PySpice and Matplotlib libraries installed
2. Computer with an internet connection to set up the Python environment

Circuit diagram:



Procedure:

1. **Setup the Python Environment:**
   - Ensure that Python, PySpice, and Matplotlib libraries are installed on your computer. If not, install them using the following commands:

     pip install PySpice matplotlib

2. **Write the Theoretical Calculations:**
   - Open a text document and write the following code to perform the theoretical calculations:

```
import numpy as np
from engineering_notation import EngNumber

# Given values
H = 70  # Heart rate in beats per minute
Vs = 6  # Supply voltage in volts
Switching_factor = 0.68
Vo = Switching_factor * Vs
C = 47e-6  # Capacitance in farads

# Calculate the required resistor value
R = -60 / (H * C * np.log(1 - (Vo / Vs)))
print('R={0}'.format(EngNumber(R)))
```

3. **Write the Circuit Code:**
    - Define the pacemaker circuit with the calculated resistor value and simulate its transient response:

```
import matplotlib.pyplot as plt
from matplotlib.widgets import Cursor
import numpy as np

import PySpice.Logging.Logging as Logging
logger = Logging.setup_logging()

from PySpice.Probe.Plot import plot
from PySpice.Spice.Netlist import Circuit
from PySpice.Unit import *

# Given values
Vs = 6@u_V
Switching_factor = 0.68
Vo = Switching_factor * Vs
C = 47@u_uF

# Calculated resistor value
H = 70  # Heart rate in beats per minute
R = -60 / (H * (C.value) * np.log(1 - (Vo.value / Vs.value)))

# Print the calculated resistor value
print('R={0}'.format(EngNumber(R)))

# Define the circuit
circuit = Circuit('Pacemaker Circuit')

steptime = 1@u_ms
finaltime = 60@u_s

# Define the switch model
circuit.model('switch', 'SW', Ron=1@u_mΩ, Roff=1@u_GΩ, Vt=2.05@u_V,
Vh=2.03@u_V)

# Define the circuit components
circuit.V(1, 1, circuit.gnd, Vs)
circuit.R(1, 1, 'control', R@u_Ω)
circuit.VoltageControlledSwitch(1, 'control', 'out', 'control', circuit.gnd,
model='switch')
circuit.C(1, 'control', circuit.gnd, C)
circuit.R(2, 'out', circuit.gnd, 50@u_Ω)

# Perform the simulation
simulator = circuit.simulator(temperature=25, nominal_temperature=25)
ic = 0@u_V
simulator.initial_condition(control=ic)
```

```
analysis = simulator.transient(step_time=steptime, end_time=finaltime)

# Plot the results
figure, axe = plt.subplots(figsize=(10, 5))

plt.title('Voltage across Capacitor')
plt.xlabel('Time [s]')
plt.ylabel('Voltage [V]')
plt.grid()
plot(analysis['control'], axis=axe, color='black')

cursor = Cursor(axe, useblit=True, color='red', linewidth=1)
plt.show()
```
Save the text document in .py format.

4. **Run the Simulation:**
    - Execute the Python script to run the simulation. This will compute the transient response of the pacemaker circuit and generate a plot showing the voltage across the capacitor.
5. **Analyze the Results:**
    - Observe the output of the script, which will display the plot of the voltage across the capacitor over time. The plot should show the periodic charging and discharging behavior corresponding to the pacemaker's operation.

Result:

EXPERIMENT-7 **Date:**

VERIFICATION OF TRUTH TABLES OF LOGIC GATES USING MULTISIM

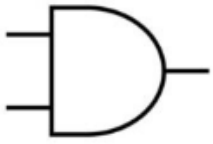
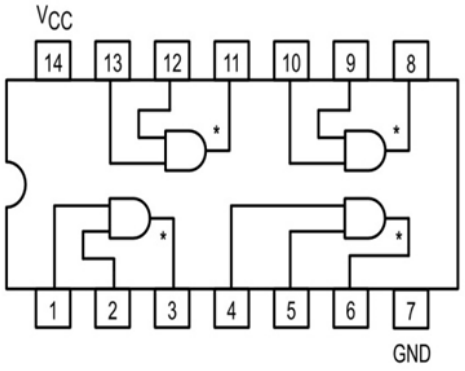
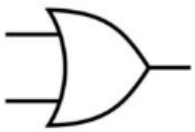
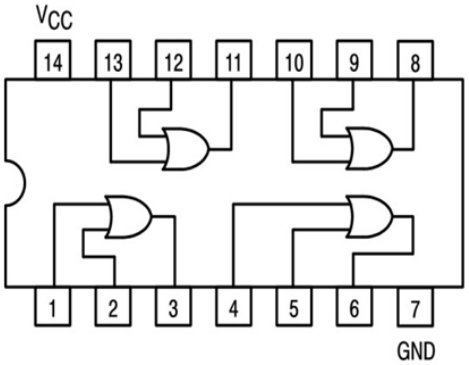
Aim:

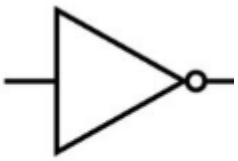
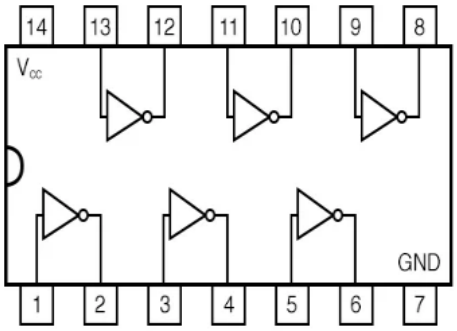
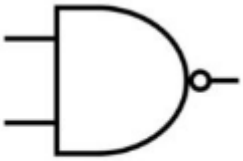
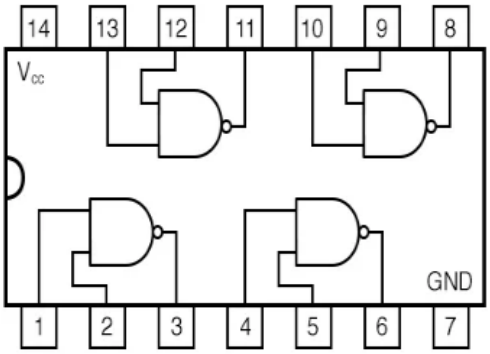
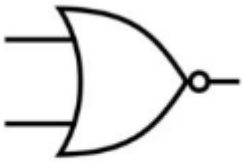
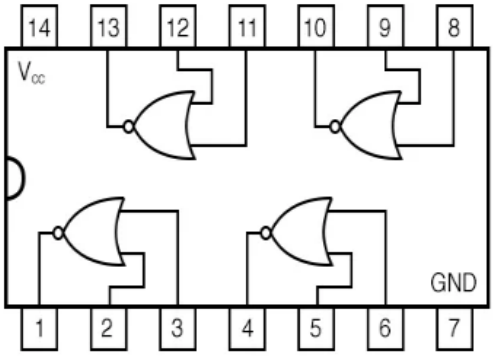
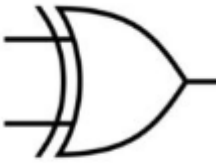
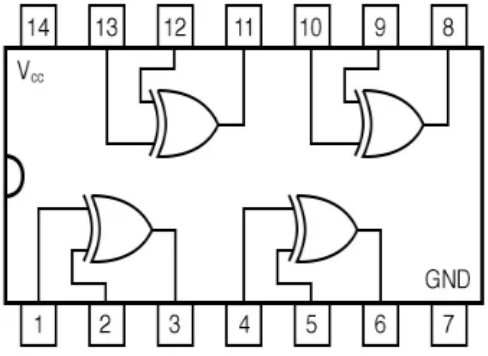
To verify the truth tables of basic logic gates (AND, OR, NOT, NAND, NOR, XOR, XNOR) using Multisim.

Apparatus Required:

- Computer with Multisim software installed
- Digital input switches (Logic Toggles)
- Digital output indicators (LEDs)
- Basic logic gates (AND, OR, NOT, NAND, NOR, XOR, XNOR)

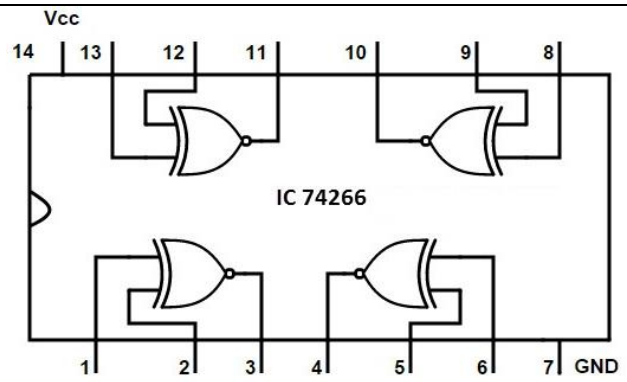
Diagram:

SYMBOL	IC-PACKAGE
 AND	 IC-7408
 OR	 IC-7432

 NOT	 IC-7404
 NAND	 IC- 7400
 NOR	 IC-7402
 XOR	 IC-7486



XNOR Gate



IC-74266

Procedure:

Open Multisim:

- Launch the Multisim software on your computer.

Create a New Project:

- Go to File > New > Project.
- Name your project and click Create.

Place Logic Gates:

- Go to the Place menu and select Component.
- In the Select a component window, choose TTL as the family.
- Select the desired logic gate (e.g., AND gate 7408, OR gate 7432, NOT gate 7404, etc.) and place them on the schematic workspace.

Add Inputs:

- Place digital input switches (logic toggles) from the component library. These will serve as the inputs to your logic gates.
- Connect the inputs to the respective pins of the logic gates.

Add Outputs:

- Place digital output indicators (LEDs) from the component library. These will serve as the outputs to display the results of the logic gates.
- Connect the outputs to the respective pins of the logic gates.

Connect Power Supply:

- Ensure that your logic gates are connected to the necessary power supply. Typically, for TTL logic gates, you will connect the Vcc and GND appropriately.

Configure Inputs:

- Set the initial states of the digital input switches to represent all possible combinations of inputs (00, 01, 10, 11 for two-input gates).

Simulate the Circuit:

- Run the simulation by clicking on the Run button.
- Observe the output indicators (LEDs) for each combination of inputs and record the results.

Verify Truth Table:

- Compare the observed outputs with the expected truth table for each logic gate.

Result:

AND Gate: For a 2-input AND gate, verify the following truth table:

Input A	Input B	Output
0	0	0
0	1	0
1	0	0
1	1	1

OR Gate: For a 2-input OR gate, verify the following truth table:

Input A	Input B	Output
0	0	0
0	1	1
1	0	1
1	1	1

NOT Gate: For a single-input NOT gate, verify the following truth table:

Input	Output
0	1
1	0

NAND Gate: For a 2-input NAND gate, verify the following truth table:

Input A	Input B	Output
0	0	1
0	1	1
1	0	1
1	1	0

NOR Gate: For a 2-input NOR gate, verify the following truth table:

Input A	Input B	Output
0	0	1
0	1	0
1	0	0
1	1	0

XOR Gate: For a 2-input XOR gate, verify the following truth table:

Input A	Input B	Output
0	0	0
0	1	1
1	0	1
1	1	0

XNOR Gate: For a 2-input XNOR gate, verify the following truth table:

Input A	Input B	Output
0	0	1
0	1	0
1	0	0
1	1	1

EXPERIMENT-8 **Date:**

SIMULATION OF HALF ADDER AND FULL ADDER CIRCUIT IN MULTISIM

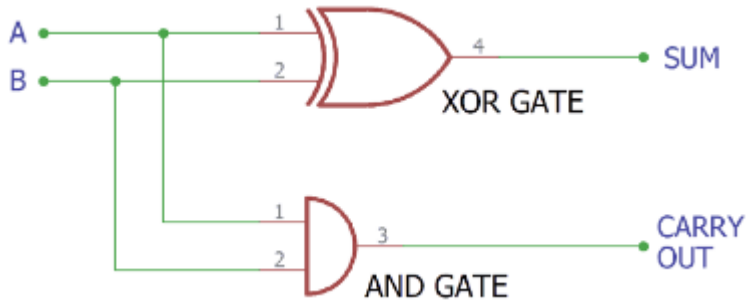
Aim:

To design and simulate the half adder and full adder circuits using Multisim and verify their truth tables.

Apparatus Required:

- Computer with Multisim software installed
- Basic logic gates (AND, OR, NOT, XOR)
- Digital input switches (Logic Toggles)
- Digital output indicators (LEDs)

Circuit diagram



Procedure:

Part 1: Half Adder

1. **Open Multisim:**
    - Launch Multisim on your computer.
2. **Create a New Project:**
    - Go to File > New > Project.
    - Name your project and click Create.
3. **Place Components for Half Adder:**
    - Select the Place menu and choose Component.
    - In the Select a component window, choose TTL as the family.
    - Select the following components and place them on the schematic workspace:
        - XOR gate (7408)

- AND gate (7432)
- Digital input switches (Logic Toggles)
- Digital output indicators (LEDs)

4. **Connect the Circuit:**
    - Connect the inputs of the XOR gate to the input switches. This gate will produce the SUM output.
    - Connect the inputs of the AND gate to the same input switches. This gate will produce the CARRY output.
    - Connect the outputs of the XOR gate and the AND gate to separate LEDs to display the SUM and CARRY respectively.
5. **Configure Inputs and Simulate:**
    - Set the input switches to all possible combinations (00, 01, 10, 11).
    - Run the simulation and observe the outputs for each combination.
6. **Verify the Truth Table:**
    - The expected truth table for the half adder is:

Input A	Input B	Sum (XOR)	Carry (AND)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

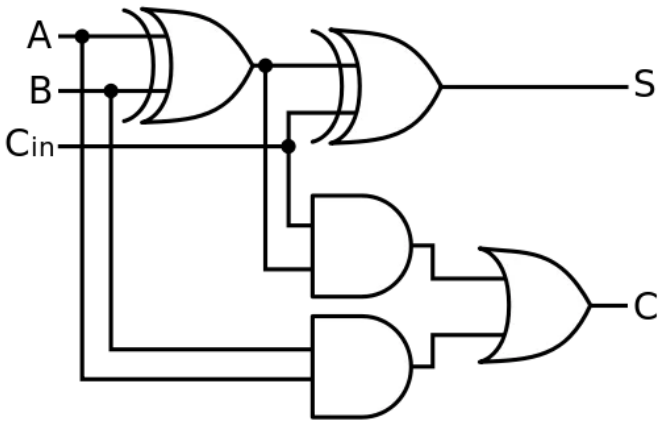
Result:

Part 2: Full Adder

1. **Open Multisim:**

   Use the same project or create a new one.

   Circuit diagram:



2. **Place Components for Full Adder:**
   - Select the Place menu and choose Component.
   - In the Select a component window, choose TTL as the family.
   - Select the following components and place them on the schematic workspace:
     - XOR gates (7408)
     - AND gates (7432)
     - OR gate (7432)
     - Digital input switches (Logic Toggles)
     - Digital output indicators (LEDs)
3. **Connect the Circuit:**
   - For the SUM output:
     - Connect two input switches (A and B) to the inputs of the first XOR gate.
     - Connect the output of this XOR gate and the third input switch (Cin) to the inputs of the second XOR gate. The output of this second XOR gate will be the SUM.

- For the CARRY output:
  - Connect inputs A and B to the inputs of the first AND gate.
  - Connect the output of the first XOR gate and Cin to the inputs of the second AND gate.
  - Connect the outputs of the two AND gates to the inputs of the OR gate. The output of this OR gate will be the CARRY.
- Connect the outputs of the second XOR gate (SUM) and the OR gate (CARRY) to separate LEDs.

4. **Configure Inputs and Simulate:**
   - Set the input switches to all possible combinations (000, 001, 010, 011, 100, 101, 110, 111).
   - Run the simulation and observe the outputs for each combination.
5. **Verify the Truth Table:**
   - The expected truth table for the full adder is:

Input A	Input B	Cin	Sum (XOR)	Carry (OR)
0	0	0	0	0
0	1	0	1	0
1	0	0	1	0
1	1	0	0	1
0	0	1	1	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	1

Result:

EXPERIMENT-9 **Date:**

IMPLEMENTATION OF BOOLEAN FUNCTION IN SOP AND POS FORM USING MULTISIM

Aim:

To implement the Boolean function XSOP=∑(1,3,6)=A′.B′.C+A′.B.C+A.B.C′ and its equivalent in the Product of Sums (POS) form XPOS=∏(1,3,6) using Multisim.

Apparatus Required

1. **Multisim Software**: For circuit simulation.
2. **Logic Gates**:
    - AND Gates (3 inputs)
    - OR Gates (3 inputs)
    - NOT Gates
3. **Digital Logic Probes**: To monitor the output.
4. **Power Supply**: To power the logic gates.

Circuit Diagram

1. **Sum of Products (SOP) Form**:
    - Use NOT gates to generate the complements A′, B′, and C′.
    - Use AND gates to implement each product term:
        - A′.B′.C
        - A′.B.C
        - A.B.C′
    - Use an OR gate to sum the outputs of the AND gates.
2. **Product of Sums (POS) Form**:
    - Implement each sum term using OR gates:
        - (A+B+C′)
        - (A+B′+C)
        - (A′+B+C)
    - Use an AND gate to multiply the outputs of the OR gates.

Procedure

1. **Set Up the Circuit in Multisim**:
    - Open Multisim and create a new project.
    - Place the required logic gates (AND, OR, NOT) on the workspace.
    - Connect the inputs A, B, and C to the gates according to the Boolean expression.
    - For SOP:
        - Connect A′, B′, and C′ to the corresponding AND gates as per the product terms.
        - Connect the outputs of the AND gates to the OR gate.
    - For POS:
        - Connect the inputs to the OR gates as per the sum terms.
        - Connect the outputs of the OR gates to the AND gate.
    - Place digital probes at the output of the final OR gate (SOP) and the AND gate (POS) to monitor the output.
2. **Run the Simulation**:
    - Assign logical values (0 or 1) to the inputs A, B, and C.
    - Run the simulation and observe the output on the digital probes.
    - Verify that the outputs match the expected results for both SOP and POS forms.
3. **Verification**:
    - Check the truth table for both implementations by changing the input combinations and ensuring that the outputs match the Boolean expression.

Truth Table

A	B	C	A'	B'	C'	A'.B'.C	A'.B.C	A.B.C'	SOP Output	A + B + C'	A + B' + C	A' + B + C	POS Output
0	0	0	1	1	1	0	0	0	0	1	1	1	1
0	0	1	1	1	0	1	0	0	1	0	1	1	1
0	1	0	1	0	1	0	1	0	1	1	0	1	1
0	1	1	1	0	0	0	0	0	0	0	0	1	0
1	0	0	0	1	1	0	0	1	1	1	1	1	1
1	0	1	0	1	0	0	0	0	0	0	1	1	0
1	1	0	0	0	1	0	0	1	1	1	0	1	1
1	1	1	0	0	0	0	0	0	0	0	0	0	0

Result:

EXPERIMENT-10 **Date:**

IMPLEMENTATION OF COMBINATIONAL CIRCUIT USING MINIMUM NUMBER OF NAND GATES USING MULTISIM

Aim

To implement the Boolean function F=AB+BC+ACF = AB + BC + ACF=AB+BC+AC using the minimum number of NAND gates in Multisim and verify the output for all possible input combinations.

Apparatus Required

1. Multisim Software: For circuit design and simulation.
2. NAND Gates:
    - 3 NAND gates for generating the AND terms ABABAB, BCBCBC, and ACACAC.
    - 1 NAND gate for combining the outputs to achieve the OR operation.
3. Logic Inputs (Switches):
    - 3 logic switches to provide binary inputs AAA, BBB, and CCC.
4. Output Indicators (LEDs):
    - 1 LED to display the final output FFF.
5. Connecting Wires: To connect the components within the Multisim workspace.

Theory

1. Express the Function in NAND-NAND Logic

To convert the expression to use only NAND gates, we need to apply De Morgan's laws and NAND equivalences:

- **Step 1:** The original expression is F=AB+BC+AC
- **Step 2:** Express OR operations using NAND gates:

  F=(AB)′·(BC)′·(AC)′
    - This step introduces a double negation, which is valid since x=(x′′)

2. Implement Using NAND Gates

1. **AND Operations Using NAND Gates:**
    - AB can be implemented as (A·B)′ using a NAND gate and then NOT the result using another NAND gate.

- BC can be implemented similarly using two NAND gates.
- AC can be implemented similarly using two NAND gates.

2. **OR Operation Using NAND Gates:**
   - The OR operation (AB)′·(BC)′·(AC)′(AB)' is realized by taking the NAND of the NAND outputs.

3. Multisim Implementation

Here is how you can construct the circuit:

Create the AND terms:

- Use two NAND gates to create each of the AND terms:
  - AB=((A·B)′)′
  - BC=((B·C)′)′
  - AC=((A·C)′)′

Combine the AND terms using NAND:

- Take the output of each AND gate, invert each using a NAND gate, then feed all three into a final NAND gate to get the OR result:
  - F=((AB)′·(BC)′·(AC)′)′

Procedure

1. **Design the Circuit:**
   - **Step 1:** Open Multisim software and start a new project.
   - **Step 2:** From the component library, select NAND gates and place them on the workspace.
   - **Step 3:** Connect the first NAND gate to inputs A and B to form the term AB.
   - **Step 4:** Connect the output of the first NAND gate to the inputs of a second NAND gate to obtain the AND operation for AB. Repeat this for the other terms BC and AC.
   - **Step 5:** Connect the outputs of the three AND terms to the inputs of a fourth NAND gate to perform the OR operation.
   - **Step 6:** Attach three logic switches to serve as inputs A, B, and C.
   - **Step 7:** Connect the output of the final NAND gate to an LED to display the output F.
   - **Step 8:** Add a power source and ground connections if required by the circuit configuration in Multisim.
2. **Simulate the Circuit:**

- **Step 1:** Run the simulation in Multisim.
- **Step 2:** Toggle the switches for inputs A, B, and C to test all possible combinations (000, 001, 010, 011, 100, 101, 110, 111).
- **Step 3:** Observe the LED to verify whether the output F is correct according to the Boolean function F=AB+BC+AC

3. **Verify the Output:**
   - **Step 1:** Record the LED output for each input combination.
   - **Step 2:** Compare the recorded results with the expected truth table for the Boolean function.

Result

After implementing the circuit and running the simulation, the output F for all input combinations was observed. The results are summarized as follows:

Inputs A	Inputs B	Inputs C	Expected Output F=AB+BC+AC	Observed Output
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	1	1
1	0	0	0	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

The observed output matches the expected output, confirming the correct implementation of the Boolean function F=AB+BC+AC.

EXPERIMENT-11 **Date:**

CHARATERISTICS OF PN-JUNCTION DIODE IN MATLAB/SIMULINK

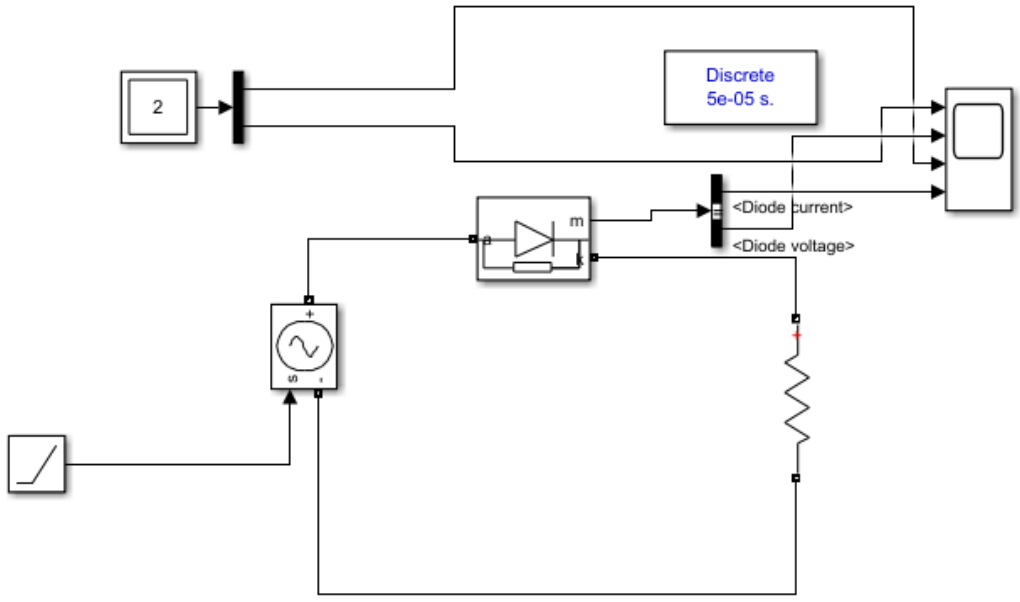
Aim

To plot and analyze the I-V characteristics of a diode using MATLAB.

Apparatus Required

1. MATLAB Software: For simulation and plotting.
2. Diode model: Standard diode model in Simulink
3. Resistor: A resistor connected in series with the diode to limit the current.
4. Controlled Voltage Source.
5. Ramp.
6. Multimeter: To measure the current and voltage through the diode.
7. Scope: To visualize the I-V characteristics of the diode.

Circuit Diagram



Procedure

1. Open MATLAB and Simulink: Start by launching MATLAB and opening Simulink.
2. Create a new Simulink model: Select "Blank Model" and create a new Simulink file.
3. Add components:
    - Drag and drop the following components:

- Controlled Voltage Source: To provide the input voltage.
- Diode: Available in the *Simscape* > *Electrical* > *Specialized Power Systems* > *Power Electronics* library.
- Resistor: To limit current.
- Ramp: To vary the voltage
- Multilmeter : To measure the voltage and current
- Scope: To display the I-V curve.

4. **Connect the circuit:**
    - Connect the components to form a circuit where the DC voltage source is connected in series with the diode and resistor. The current and voltage measurement blocks should be connected in series with the diode and to the scope for visualization.
5. **Run the simulation:**
    - Set the DC voltage source to sweep across a range of voltages (e.g., -1V to 1V).
    - Run the simulation and observe the I-V characteristics on the scope.
6. **Analyze the I-V curve:**
    - The scope will show the voltage versus current curve of the diode, which represents its I-V characteristics.

Result

The I-V characteristics of the diode follow an exponential relation for forward bias and almost negligible current for reverse bias.

EXPERIMENT-12 **Date:**

DESIGN A VOLTAGE REGULATOR USING ZENER DIODE IN MATLAB/SIMULINK

Aim

To demonstrate and analyse the behaviour of a Zener diode as a voltage regulator and to observe how the Zener diode maintains a constant output voltage across a load despite variations in the input voltage.

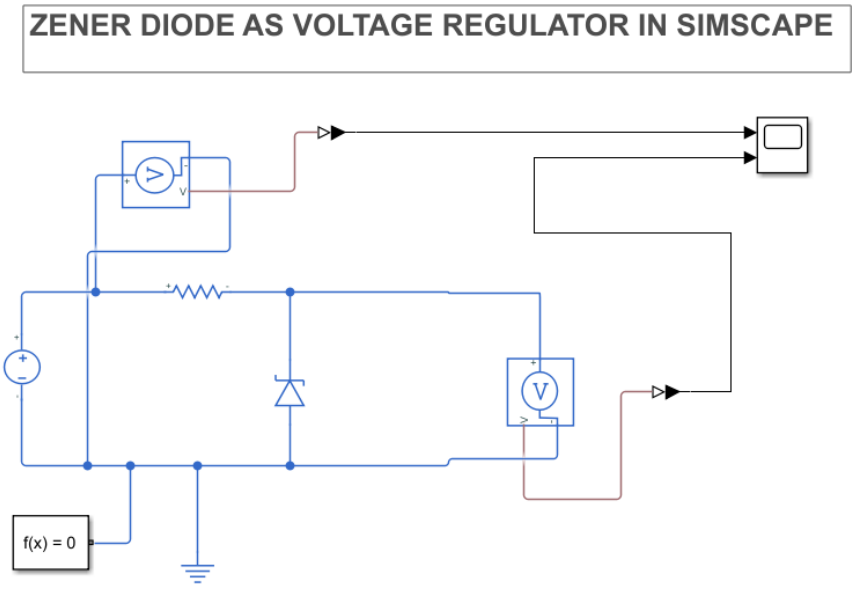
Apparatus Required

1. MATLAB with Simulink installed.
2. Components in Simulink:
    - DC Voltage Source
    - Zener Diode
    - Resistor (Current-limiting)
    - Ground
    - Voltage Sensor
    - Scope for waveform observation

Procedure

1. **Setup the Circuit in Simulink**:
    - Open MATLAB and launch Simulink.
    - Create a new blank model and name it.
2. **Add Components**:
    - Go to *Simscape* > *Foundation Library* > *Electrical* > *Electrical Sources* and drag a **DC Voltage Source** to the model.
    - From *Simscape* > *Foundation Library* > *Electrical* > *Electrical Elements*, drag a **Resistor** to represent the current-limiting resistor.
    - From the same library, drag a **Diode** and set its parameters to represent a Zener diode (set breakdown voltage).
    - Add a **Voltage Sensor** from *Simscape* > *Foundation Library* > *Electrical* > *Sensors*.
    - Drag a **Scope** to observe the output voltage.
    - Don’t forget to add **Ground** from *Simscape* > *Foundation Library* > *Electrical* > *Electrical Elements*.

3. **Connect Components**:
    - Connect the **DC Voltage Source** in series with the **Current-limiting Resistor** and the **Zener Diode**.
    - Connect the **Voltage Sensor** across the load resistor to measure the output voltage.
    - Attach the **Scope** to the output of the Voltage Sensor to observe the waveforms.
4. **Set Parameters**:
    - Set the **DC Voltage Source** to a value higher than the Zener breakdown voltage (e.g., 15V for a Zener diode with a breakdown voltage of 5V).
    - Set appropriate value for the current-limiting resistor (e.g., 1kΩ)
    - Set the Zener diode breakdown voltage to a typical value (e.g., 5.1V).
5. **Circuit Diagram**



6. **Run the Simulation**:
    - Set the simulation time (e.g., 1 seconds).
    - Run the simulation to observe the output voltage across the load.
7. **Observation**:
    - Observe the waveform in the **Scope** block. You should see that the voltage across the load remains steady (close to the Zener diode breakdown voltage), even if the input voltage is varied.
8. **Result**

EXPERIMENT-13 **Date:**

SIMULATION OF HALFWAVE RECTIFIER IN MATLAB/SIMULINK

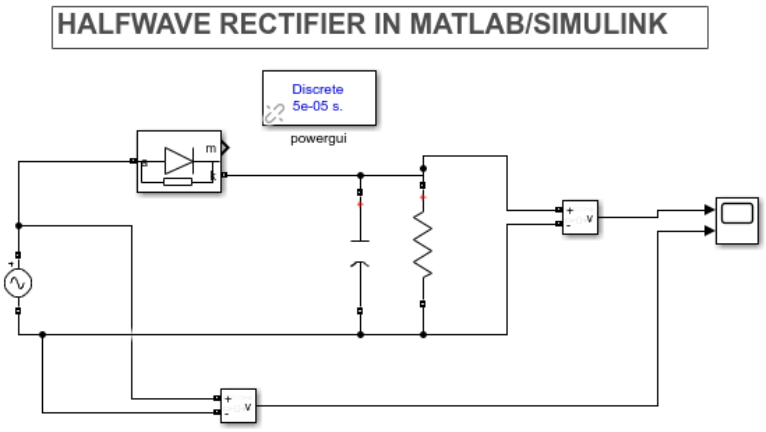
Aim:

To simulate the operation of half-wave rectifier with and without filter circuits in MATLAB/Simulink and analyse their output waveforms.

Apparatus Required:

- MATLAB/Simulink software installed on a computer.
- Diode – for rectification.
- AC Voltage Source – to supply the input signal.
- Resistive Load (R) – to represent the load for the rectifier.
- Capacitor (optional) – for filtering the output (to smooth the ripple).
- Scope block – to observe the waveforms.
- Voltage Measurement Blocks – to measure input and output voltages.
- Current Measurement Blocks – to measure load current.
- Power GUI block – for simulation control.

Circuit Diagram



Procedure:

1. Simulation of Half-Wave Rectifier (Without Filter):

Step 1: Open MATLAB/Simulink and create a new model.

Step 2: Drag and drop the following components from the Simulink library:

- AC Voltage Source (set to peak or RMS value as per requirement).
- Diode (ideal or real model).

- Resistive Load (R).
- Ground block (to ensure proper simulation).
- Scope block (to observe waveforms).

Step 3: Connect the components as per the circuit diagram of a half-wave rectifier:

- Connect the positive terminal of the AC source to the anode of the diode.
- Connect the cathode of the diode to one end of the load resistor.
- Connect the other end of the resistor to ground.
- Connect the negative terminal of the AC source to the ground.
- Attach the Scope block at the output (across the load resistor) to visualize the rectified output waveform.

Step 4: Set simulation parameters:

- Run the simulation for at least 2 to 3 cycles of the AC waveform (adjust simulation time accordingly).

Step 5: Observe the waveform of the rectified signal on the Scope block.

- You will observe a pulsating DC output with a large ripple.

2. Simulation of Half-Wave Rectifier (With Filter):

Step 1: In the previous half-wave rectifier circuit, add a Capacitor across the load resistor (R).

Step 2: Adjust the capacitance value depending on the desired smoothing effect (refer to capacitance calculation using formulae $C = \frac{I_{Load}}{f*\Delta V}$.

Step 3: Run the simulation and observe the output waveform on the Scope block.

- The output should now be a smoothed DC signal with less ripple compared to the unfiltered output.

Result

EXPERIMENT-14 **Date:**

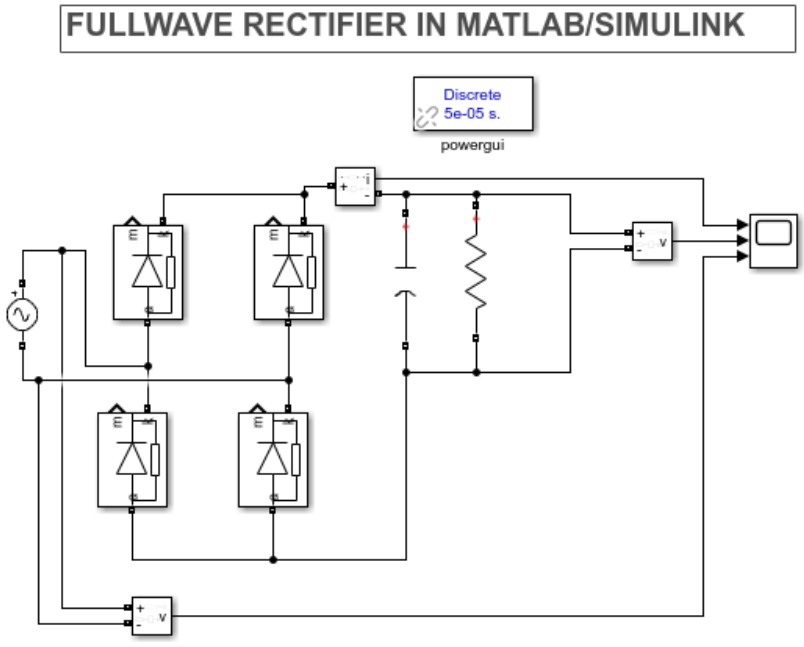
SIMULATION OF FULLWAVE RECTIFIER IN MATLAB/SIMULINK

Aim: To simulate the operation of full-wave rectifier with and without filter circuits in MATLAB/Simulink and analyse their output waveforms.

Apparatus Required:

- MATLAB/Simulink software installed on a computer.
- Diode – for rectification.
- AC Voltage Source – to supply the input signal.
- Resistive Load (R) – to represent the load for the rectifier.
- Capacitor (optional) – for filtering the output (to smooth the ripple).
- Scope block – to observe the waveforms.
- Voltage Measurement Blocks – to measure input and output voltages.
- Current Measurement Blocks – to measure load current.
- Power GUI block – for simulation control.

Circuit Diagram



Procedure:

1. Simulation of Full-Wave Rectifier (Without Filter):

Step 1: Create a new Simulink model for the Full-Wave Rectifier (Bridge Configuration).

Step 2: Drag and drop the following components:

- AC Voltage Source,
- 4 Diodes (for bridge rectification),
- Resistive Load (R),
- Scope block,

Step 3: Connect the components as per the bridge rectifier circuit:

- Connect the AC source to the diagonal input of the diode bridge (2 opposite corners).
- Connect the load resistor across the other diagonal of the bridge (diodes' cathode and anode junction).
- Attach the Scope block at the output (across the load) to visualize the rectified output.

Step 4: Run the simulation for 2 to 3 cycles and observe the output waveform.

- You will observe a pulsating DC output but with both positive and negative halves of the AC cycle being rectified, reducing ripple compared to the half-wave rectifier.

2.Simulation of Full-Wave Rectifier (With Filter):

Step 1: In the previous full-wave rectifier circuit, add a Capacitor across the load resistor.

Step 2: Set an appropriate capacitance value for smoothing the ripple.

Step 3: Run the simulation and observe the output waveform.

- The output should now be a smoother DC with even less ripple compared to the unfiltered full-wave rectified output.

Result